Clima Hermosillo



Recolección, almacenamiento y visualización de datos meteorológicos de Open-Meteo.

PYTHON · POSTGRESQL · STREAMLIT · DOCKER

UBICACIÓN FIJA

29.0729° N · 110.9559° O

America/Hermosillo · UTC−7

Objetivo del proyecto

Consultar una API externa, guardar sus datos y mostrarlos en un dashboard.

PASO 1

Consultar

Condiciones actuales estimadas y pronósticos para una ubicación fija.

PASO 2

Conservar

Capturas periódicas y versiones de los pronósticos dentro de PostgreSQL.

PASO 3

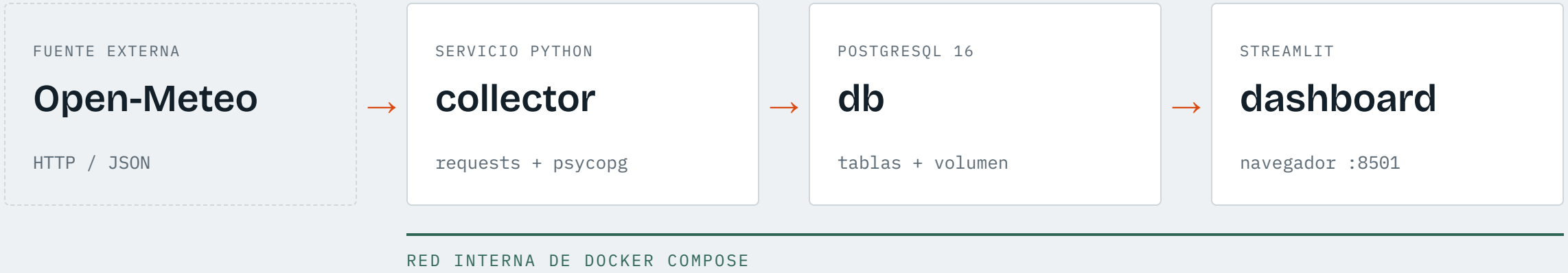
Comparar

Diferencias entre valores actuales y predicciones hechas con un día de anticipación.

El proyecto consume predicciones externas; no entrena un modelo propio.

Arquitectura

Tres contenedores en Docker Compose. La recolección y la visualización son independientes.



Proceso de recolección

Coordenadas fijas y zona horaria America/Hermosillo.

01	**Consultar**	Dos peticiones: clima actual y predicciones anteriores.
02	**Transformar**	Fechas, series horarias y alineación de valores.
03	**Guardar**	Tres tablas en una sola transacción de PostgreSQL.
04	**Esperar**	600 segundos al terminar; después, repetir.

Timeout HTTP de 30 s. Si ocurre un error, se registra y se reintenta en el siguiente ciclo.

Modelo de datos

Identificadores e índices temporales; sin claves foráneas entre tablas.

observations

¿Qué valor tenemos ahora?

- fetched_at · observed_at
- temperature · humidity
- apparent_temperature
- precipitation · wind_speed

forecasts

¿Qué esperábamos para cada hora?

- fetched_at · forecast_for
- temperature · precipitation
- precipitation_probability
- UNIQUE (captura, hora destino)

comparisons

¿Cuánto cambió la predicción?

- fetched_at · valid_hour
- current_temperature
- predicted_temperature_24h
- lluvia actual y predicha

Versiones de pronóstico

Cada descarga se guarda como una versión distinta del pronóstico.

`fetched_at`

Momento en que se descargó el pronóstico.

`forecast_for`

Hora para la que se calculó la predicción.

Hoy, 10:00
Hora de descarga

→

Mañana, 15:00
Hora pronosticada

→

32 °C
Predicción · ejemplo ilustrativo

Comparación de pronósticos

Valor actual contra el pronóstico hecho 24 horas antes para la misma hora.

Actual estimado − Pronóstico de hace 24 h = Desviación

34 − 32 = +2 °C

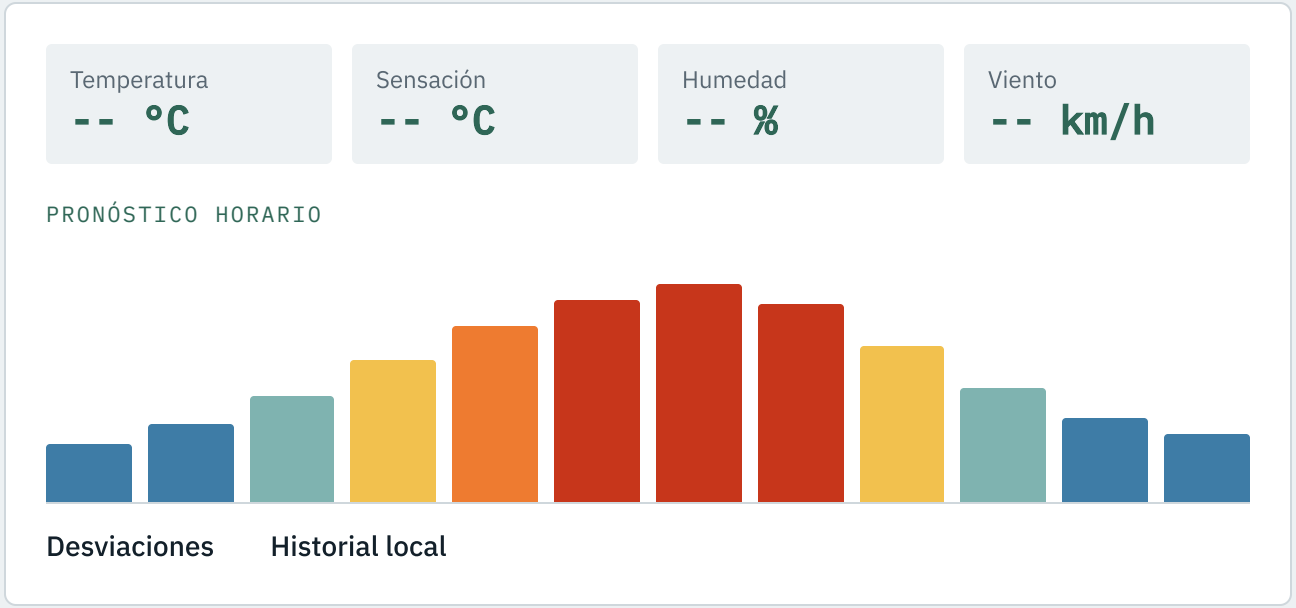
Ejemplo ilustrativo, no una captura del sistema.

Se comparan estimaciones de modelos. Para medir precisión real se necesitan observaciones independientes.

La precipitación se compara por hora para no mezclar intervalos distintos.

Dashboard

Esquema de la interfaz; no representa datos reales.



LECTURA

Desde PostgreSQL

Última observación, última versión del pronóstico y hasta 300 capturas.

60 s

Intervalo de actualización de la vista.

Configuración con Docker

Servicios, imágenes y variables definidas en compose.yaml.

SERVICIO	IMAGEN	CONFIGURACIÓN PRINCIPAL
db	postgres:16-alpine	Volumen postgres_data + schema.sql al iniciar
collector	python:3.12-slim	DATABASE_URL + UPDATE_SECONDS=600
dashboard	python:3.12-slim	Puerto 8501 + DATABASE_URL

db:5432 es una dirección interna. El navegador entra por localhost:8501.

Operación y manejo de errores

Arranque, reintentos y persistencia de datos.

ARRANQUE

Base disponible

`pg_isready` valida PostgreSQL antes de iniciar los otros servicios.

RECUPERACIÓN

Errores registrados

El recolector registra excepciones y vuelve a intentar después de la espera.

PERSISTENCIA

Volumen de datos

`down` conserva los datos; `down -v` también los elimina.

TRANSACCIONES

Escritura transaccional

Una falla de escritura revierte las inserciones de ese ciclo.

Limitaciones

Puntos a considerar al interpretar los resultados y para trabajo futuro.

Horizonte del pronóstico

Tres días solicitados no garantizan 72 horas futuras completas.

Comparación temporal

La temperatura actual puede no coincidir con la hora redondeada.

Dependencia externa

Si falla la segunda API, tampoco se guarda la captura actual.

Paso a producción

Gestionar secretos, migraciones, retención y monitoreo.

Demostración

Iniciar los servicios antes de la exposición.

01 · INICIAR	`$ docker compose up --build -d`
02 · REVISAR	`$ docker compose ps`
03 · ACTIVIDAD	`$ docker compose logs --tail=30 collector`
04 · NAVEGAR	`→ http://localhost:8501`